PAGE NO :
DATE :

① What are constructors? Explain 2 types of constructors.

→ * A constructor initializes an object immediately upon creation. It has same name as the class in which it resides and is syntactically similar to method. Once defined, the constructor is automatically called immediately after the object is created, before the new operator completes.

* They do not have return type, not even void.

Types of constructors

① Default constructor.

* It is a constructor that takes no parameters.
* If you don't explicitly define a constructor in a class, Java automatically provides a default one

Example

```
public class Car {
    car() {
        System.out.println("Default constructor
                            called");
    }
    public static void main (String[] args) {
        Car myCar = new Car();
    }
}
```

② Parameterized constructor

* It is a constructor that takes arguements to initialize the object with specific values.

Example:

```
public class Car {
    String model;
```

DATE :

```
Car (String m) {
  model = m ;
}
public static void main(String[] args) {
   Car myCar = new
Car ("Tesla");
  System.out.println ("Model: " + myCar.model);
}
}
```

② Write a Java program demonstrating Method overloading

=> Method overloading means defining multiple methods in the same class with the same name but different parameters (typ, number or both)

```
public class OverloadExample {
   void show (int a) {
    System.out.println ("Number: "+a) ;
}
   void show (String name) {
      System.out.println ("Name: " + name);
}
    public static void main(String[]args) {
       OverloadExample obj = new OverloadExam
                              -ple() ;
       obj.show (100);
       obj.show ("Alice");
    }
}
```

Output: Number : 100
Name : Alice.

PAGE NO:
DATE:

6. Define recursion. Write a recursive program to find nth Fibonacci number

* Recursion in Java is a process in which a method calls itself to solve a smaller part of the original problem.

```
class Fibonacci {
    static int fib(int n) {
        if (n<= 1)
            return n;
        return fib(n-1) + fib(n-2);
    }
    public static void main(String[] args) {
        int n=10;
        System.out.println("Fibonacci Series:");
        for(int i=0; i<n; i++) {
            System.out.print (fib(i)+" ");
        }
    }
}
```

Output:

Fibonacci Series:
0 1 1 2 3 5 8 13 21 24.

④ Explain the various access specifiers in Java.

* Access specifiers define the visibility or access level of classes, methods and variables

① Private

* Accessible only within the same class.
* Not visible to other classes or subclasses

Example :

```
class Example {
    private int num = 10 ;
    private void show () {
     System.out.println ("Private method");
    }
}
```

② Protected

* Accessible within the same package and by subclasses in other packages.
* Often used in inheritance.

```
class Example {
    protected int num = 30 ;
    protected void show() {
    System.out.println ("Protected method");
    }
}
```

③ Public

* Accessible from: Anywhere (any class, any package)
* Usage: when you want your method, class, or variable to be universally accessible

Example:

```
public class Example {
    public int number = 10;
     public void show () {
    System.out.println ("Public method");
    }
}
```

PAGE NO :
DATE :

⑥ Explain different types of inheritance or Explain the concept of inheritance in programming

=> Inheritance is a concept that allows a child class to acquire the properties and behaviour of a parent class.

① Single inheritance

* A sub class inherits from one super class
* This is the most basic form of inheritance in Java.

Example :

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}
```

② Multilevel inheritance

* A class inherits from a subclass which in turn inherits from another class, forming a chain.

Example :

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}
```

```
class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("The puppy weeps.");
    }
}
```

③ Hierarchical Inheritance.

* Multiple subclasses inherit from a single parent class.

Example:

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("The cat meows.");
    }
}
```

PAGE NO :
DATE :

⑥ Demonstrate two uses of super keywords with example.

① Use 1: To call the Superclass constructor.
You use super() to explicitly call the constructor of the parent class. This is especially useful when the superclass has a parameterized constructor.

② Use 2: To access superclass methods or variables
If the subclass overrides a method or has a variable with the same name, you can use super to refer to the superclass version.

Example:

```
class Animal {
  Animal() {
  System.out.println("Animal constructor
                      called");
  }
    void sound() {
  System.out.println("Animal makes a
                      sound");
  }
}
class Dog extends Animal {
    Dog() {
    super();
System.out.println("Dog constructor
                    called");
}
  void sound() {
    super.sound();
```

```
        System.out.println("Dog barks");
    }
}

public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Output:

Animal constructor called.
Dog constructor called.
Animal makes a sound
Dog barks.

⑦ Illustrate multilevel heirarchy using an example

⇒ Multilevel inheritance is when a class inherits from a subclass, which in turn inherits from another class. This forms a chain of inheritance.

class A (Parent)
↓
class B (Child of A)
↓
class C (Child of B)

Example:

```
class Animal {
    void eat() {
        System.out.println("This animal
                eats food.");
```

```
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("The puppy weeps.");
    }
}

public class Test {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.weep();
    }
}
```

Output:

```
This animal eats food.
The dog barks
The puppy weeps
```